

Sujet $\mathcal{H}$

□ Exercice : type A

On considère un scrutin avec m candidats numérotés de 0 à $m - 1$ et n votes exprimés. Chaque vote est représenté par un entier indiquant le candidat choisi. L'objectif est de déterminer si un candidat a obtenu une majorité *absolue*, c'est-à-dire strictement plus de la moitié des voix. La sortie doit être :

- l'indice du candidat ayant la majorité absolue, si un tel candidat existe ;
- -1 sinon.

Par exemple,

- avec $m = 3$ et $n = 5$, si le tableau de votes est $\{1, 0, 2, 0, 0\}$, on renvoie 0 car le candidat 0 obtient 3 voix sur 5 ;
- avec $m = 3$ et $n = 5$, si le tableau de votes est $\{1, 0, 2, 1, 0\}$, on renvoie -1 car aucun candidat n'a plus de 2 voix ;
- avec $m = 3$ et $n = 6$, si le tableau de votes est $\{1, 0, 1, 2, 1, 2\}$, on renvoie -1 car le candidat 1 a 3 voix sur 6 et n'a pas la majorité absolue.

Afin de résoudre ce problème on propose l'algorithme suivant :

- initialiser un tableau `voix` de taille m à 0,
 - pour chaque vote v incrémenter `voix[v]`,
 - tester si un candidat possède ou non la majorité absolue
1. Donner la complexité en temps et en espace de ce premier algorithme en fonction de n et m .

On s'intéresse maintenant à l'algorithme suivant proposé par Boyer-Moore en 1981 :

Algorithme : Vote majoritaire absolue

Entrées : Un tableau de n entiers tous compris entre 0 et $m - 1$

Sorties : -1 si aucun entier n'a la majorité absolue sinon l'entier apparaissant plus de $n/2$ fois.

```

1  cpt ← 0
2  cm
3  pour i ← 0 à n - 1 faire
4      si cpt = 0 alors
5          cm ← tab[i]
6          cpt ← 1
7      sinon si cm = tab[i] alors
8          cpt ← cpt + 1
9      sinon
10         cpt ← cpt - 1
11  cpt ← 0
12  pour i ← 0 à n - 1 faire
13      si tab[i] = cm alors
14          cpt ← cpt + 1
15  si cpt > n/2 alors
16      return cm
17  sinon
18      return -1

```

Cet algorithme parcourt une première fois le tableau en maintenant à jour une variable cm contenant le numéro d'un candidat *potentiellement* majoritaire (de façon absolue) puis une seconde fois afin de vérifier si ce candidat a bien la majorité absolue ou non.

2. Quelles sont les complexités en temps et en espace de ce nouvel algorithme ?
3. Prouver l'invariant suivant valable pour la première boucle **for** : « Les éléments du sous tableau $\{\text{tab}[0], \dots, \text{tab}[i-1]\}$ se décomposent en cpt exemplaires de cm et de paires formées chacune de deux éléments distincts ». On notera bien que les paires d'éléments distincts peuvent éventuellement contenir des occurrences de cm .
4. En déduire que si un candidat a la majorité absolue, alors ce candidat ne peut être que cm , conclure quant à la correction de cet algorithme.

□ Exercice : *type B*

Le tri rapide (*quicksort*) est un algorithme de tri inventé par C. Hoare en 1961. L'algorithme est généralement utilisé sur des tableaux mais peut-être adapté aux listes. Il consiste pour trier une liste `lst`, à choisir un élément `p` comme pivot puis à séparer la liste privée du pivot en deux listes : celle des éléments plus petits ou égaux au pivot et celle des éléments plus grand que le pivot. On trie ensuite récursivement ces deux sous listes puis on les concatène en incluant le pivot entre les deux.

Par exemple, en supposant que le pivot choisi est le premier élément de la liste. Pour trier `[4; 2; 7; 4; 1; 5]` on sépare cette liste en `[2; 1; 4]` et `[7; 5]` qu'on trie récursivement la concaténation (en incluant le pivot) donne alors `[1; 2; 4; 4; 5; 7]`.

1. Dans quelle famille d'algorithme peut-on ranger le tri rapide?
2. Donner en la justifiant brièvement la complexité de cet algorithme.
3. Ecrire en OCaml une fonction `separe : int list -> (int* int list * int list)` qui prend en argument une liste et la sépare en utilisant son premier élément comme pivot puis renvoie le pivot et les deux listes obtenues.
4. Ecrire en OCaml une fonction `qsort : int list -> int list` qui implémente le tri rapide.